SECURITY

SQL Injection Prevention Visual Guide

The oldest web attack still works — here's how to kill it for good.

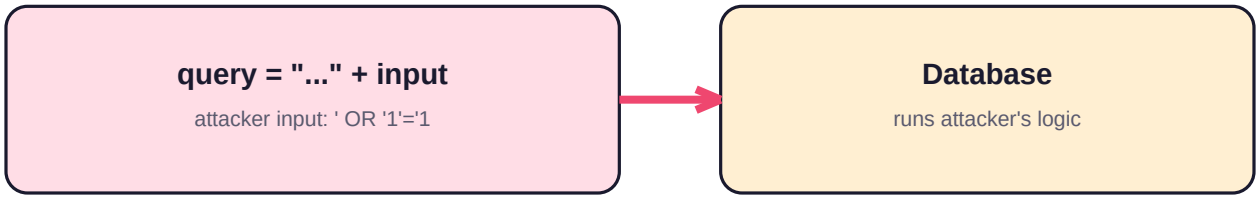
ANIMATED WALKTHROUGH

Step through pages to play the animation

STEP 1/6

Injection Mechanics

User input becomes code when you concatenate strings.

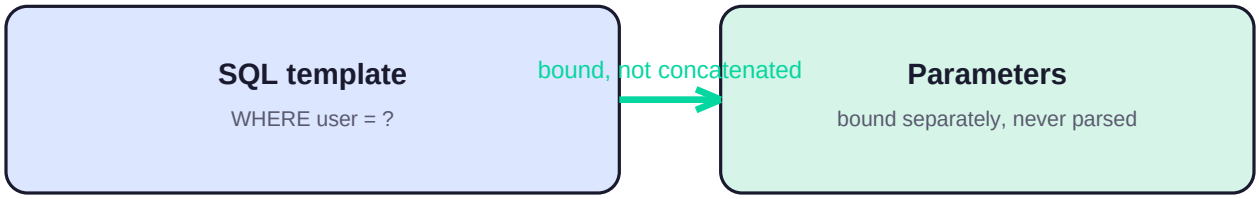


The database can't tell your SQL apart from the attacker's — it's one string by the time it arrives. The fix: never build SQL by concatenation.

STEP 2/6

Parameterized Queries

Code and data travel separately — injection becomes impossible.



Placeholders (?, :name, $1) keep input as pure data. This single practice eliminates ~all classic SQLi.

STEP 3/6

ORM Safety & Raw Queries

ORMs help — until you bypass them.

- ORM query builders parameterize automatically — prefer them over raw SQL.
- Danger zone: raw()/execute() with f-strings or % formatting — audit every use.
- Even with ORMs, validate sort/filter field names against an allowlist (no dynamic column names from input).
- Stored procedures help only if they also use parameters internally.

STEP 4/6

Least-Privilege DB Users

Assume injection happens — limit the blast radius.

App user
SELECT/INSERT/UPDATE only

Never
no DROP/ALTER, no DBA, no file read

Migrations
separate privileged user

An injected query can only do what the DB user can do. A read-only reporting user can't drop tables.

STEP 5/6

Defense in Depth

Layers around the primary fix.

- WAF with SQLi rules: catches noisy probes, not a substitute for parameters.
- Input validation: type, length, format checks — defense, not the fix.
- Error handling: generic errors to users; detailed errors only in logs (no schema leaks).
- Monitoring: alert on SQL error spikes and suspicious query patterns.

STEP 6/6

Testing Checklist

Prove it's dead, don't assume.

- SAST rules flag string-concatenated SQL in CI — break the build.
- DAST/pentest with SQLi payloads against every input: forms, headers, APIs.
- Code review: grep for raw SQL execution; each hit needs a justification.
- Re-test after refactors — one 'quick' raw query reopens the hole.

STEP ✓/✓

Key Takeaways

SQL Injection — Remember

- Injection happens when input is concatenated into SQL strings.
- Parameterized queries separate code from data — the definitive fix.
- ORMs are safe until you drop to raw SQL; audit every raw() call.
- Least-privilege DB users contain the damage if injection ever lands.
- Add WAF, validation, and safe errors as layers — not substitutes.
- Test continuously: SAST in CI, DAST payloads, and code review.

Keep exploring

Pair this guide with the matching video walkthroughs and hands-on labs at interstitiumlabs.dev